

Eco-Matic WPF Smart Vending System Documentation

1. Project Overview

Purpose

Eco-Matic is a smart vending machine system that combines product purchasing, recycling rewards, admin inventory management, sales reporting, and RFID-assisted customer accounts. The project is designed to show how a desktop application can coordinate user interface logic, cloud database access, and physical hardware feedback in one working system.

The project is also motivated by a local environmental concern. In busy urban areas such as Cebu City, improper waste disposal and visible litter remain common problems in public spaces. Eco-Matic responds to that issue by turning recycling into a simple incentive: customers can recycle eligible items, earn eco-credits, and use those credits toward vending purchases. This connects everyday buying behavior with responsible waste handling.

Sustainability Motivation

Eco-Matic supports the idea behind Sustainable Development Goal 12: Responsible Consumption and Production. Instead of treating bottles and cans only as trash, the system encourages users to return recyclable materials by giving them practical value through eco-credits. The goal is not only to sell items, but also to promote cleaner surroundings and more responsible consumption habits.

Features

- Customer vending screen with 12 visible product slots
- Machine selection before purchase
- Cash and QR-paid balance purchasing flow
- RFID customer registration and eco-credit saving
- Eco-point payment support for registered RFID customers
- **Recycling incentive flow aligned with SDG 12:** Responsible Consumption and Production
- Global item catalog management
- Per-machine slot inventory, stock, capacity, and optional item price overrides
- Admin dashboard with machines, users, customers, logs, inventory, and sales reports

- Receipt display and receipt printing support
- Arduino RFID/LCD/LED communication through USB serial
- Supabase PostgreSQL backend for persistent data

Target Audience

The system is intended for:

- vending machine operators who need inventory and sales visibility
- customers who want a simple vending flow with optional recycling rewards
- students and community users who can be encouraged to recycle through small purchase incentives
- administrators who manage machines, items, users, customers, and reports
- classroom evaluators reviewing object-oriented design, database design, and hardware integration

Technology Stack

- C# and .NET 10.0
- WPF desktop application
- Supabase PostgreSQL and PostgREST
- Supabase Edge Function for QR payment simulation
- Arduino Uno/Nano with RC522 RFID reader and I2C LCD
- WebView2 for the map picker
- QRCode for QR code generation
- System.IO.Ports for serial communication
- System.Speech for voice/audio support

2. Requirements

Software Requirements

- Windows operating system
- .NET 10 SDK

- Visual Studio or another .NET-capable IDE
- Supabase project with the required schema migrations
- Internet connection for Supabase, QR payment simulation, and map reverse-geocoding
- Repo-root .env file based on .env.example

Hardware Requirements

The software can be reviewed without hardware, but the full demo uses:

- Arduino Uno/Nano
- RC522 RFID reader
- 16x2 I2C LCD
- red/green LEDs
- USB cable connected to the configured COM port

Default serial settings:

```
COM5
9600 baud
```

Installation Steps

1. Clone the repository.
2. Copy .env.example to .env.
3. Fill in the Supabase URL and anon/publishable key.
4. Apply the Supabase SQL migrations in numeric order from docs/sql/migrations/supabase/.
5. Apply docs/sql/seeds/seed_inventory.sql if starting sample data is needed.
6. Flash Arduino/RFID_Scanner/RFID_Scanner.ino to the Arduino if hardware will be used.
7. Build and run the application:

```
dotnet build
dotnet run --project Eco-Matic.csproj
```

System Requirements and Limitations

- Customer, admin, RFID, receipt persistence, and report features require live Supabase connectivity.
- The customer vending UI supports 12 visible slots.
- Product images are loaded locally from the repository assets.
- Arduino hardware feedback depends on the correct COM port and baud rate.

3. Data and Backend Overview

Data Types and Purpose

Eco-Matic uses Supabase PostgreSQL as the main persistent backend.

Important tables include:

- roles - staff role definitions
- users - admin and inventory-manager accounts
- user_machine_assignments - machine access for inventory managers
- vending_machines - machine identity, address, and optional coordinates
- items - global product catalog
- machine_inventory - per-machine slot assignment, stock, capacity, and item price overrides
- sales_transactions - sales records for reporting
- event_logs - audit/history records
- customers - RFID customer accounts and eco-credit balances
- receipt_sessions and receipt_session_lines - detailed receipt history
- recyclable_items - configurable recycle materials and point values
- qr_payment_intents - QR payment simulation records

Data Operations

The WPF windows do not write SQL directly. Data access is routed through service classes:

- SupabaseStore handles application-level CRUD, reports, logs, receipts, and customer operations.
- SupabaseClient wraps the low-level Supabase REST calls.
- SupabaseSessionCoordinator centralizes customer-mode Supabase checks and persistence.
- DataStore keeps the active customer vending session in memory while writes are sent to Supabase.

Common operations include:

- loading machines and inventory
- adding, editing, soft-deleting catalog items
- assigning products to machine slots
- updating stock after purchase or restock
- recording sales and event logs
- saving customer eco-credit balances
- saving receipt sessions and receipt line items

Local Files and Assets

Local files still have a role in the project:

- `.env` stores local runtime configuration and is not committed.
- `.env.example` documents required environment variables.
- `Assets/Images/` stores product images.
- `Assets/Gifs/` stores README demo recordings.
- `Arduino/` stores the RFID scanner firmware.
- `docs/sql/` stores schema migrations and seed scripts.

Error Handling

The system handles common runtime issues by:

- stopping startup when required Supabase environment values are missing
- showing connectivity messages when Supabase-dependent features cannot run
- validating inventory slot ranges and duplicate slot assignments before writing
- using fallback image loading when product image paths are missing or invalid
- sending VALID, INVALID, or clear LCD/status messages back to Arduino after RFID scans

4. Code Structure

Main Program Structure

The project follows a practical MVVM-lite structure:

- WPF .xaml files define screens and layout.
- .xaml.cs files contain window behavior and event handling.
- Service classes contain backend, hardware, payment, printing, and image logic.
- Model classes represent vending items, products, transactions, and recyclable definitions.

Key Classes and Purposes

- MainWindow - application entry screen, mode routing, and RFID event handling
- CustomerWindow - customer vending, payment, recycle, and receipt flow
- AdminWindow - admin dashboard, inventory, catalog, users, machines, logs, and reports
- DataStore - in-memory customer vending session state
- SupabaseStore - main application data service
- SupabaseClient - low-level Supabase REST client
- SupabaseSessionCoordinator - customer-mode Supabase routing and availability checks
- ArduinoService - serial communication with RFID/LCD hardware
- QrPaymentService - QR payment intent creation and status checking
- ReceiptPrinterService - receipt printing route selection
- EscPosReceiptFormatter - receipt print formatting
- ImageLoader - safe image loading and fallback behavior
- VendingItem, Product, SnackItem, DrinkItem, MiscItem - product model and inheritance structure
- Transaction, TransactionItem, RecycleEntry - customer session and receipt data

Code Walkthrough

A useful walkthrough order is:

8. MainWindow.xaml.cs for startup, customer/admin routing, and RFID handling
9. CustomerWindow.xaml.cs for the purchase, recycle, QR, and receipt flow
10. AdminWindow.xaml.cs for management and reporting features
11. Data/DataStore.cs for active customer session state
12. Data/SupabaseStore.cs for backend operations

13. Data/SupabaseClient.cs for REST communication
14. Data/ArduinoService.cs for hardware communication
15. Models/Product.cs and Models/VendingItem.cs for OOP product structure
16. Models/Transaction.cs for receipt/session modeling

Modularity and Reusability

The project separates responsibilities so that major concerns are easier to maintain:

- UI windows handle interaction and display.
- Backend logic stays in SupabaseStore and SupabaseClient.
- Hardware logic stays in ArduinoService.
- QR payment logic stays in QrPaymentService.
- Receipt printing logic stays in ReceiptPrinterService and EscPosReceiptFormatter.
- Product and transaction concepts stay in model classes.

This structure prevents the UI from directly owning every backend and hardware detail.

5. User Interface

Design and Usability

The customer interface focuses on fast vending actions. It uses a 12-slot layout, clear item display, payment controls, recycle point controls, and receipt output.

The admin interface is organized by responsibility:

- Dashboard
- Inventory
- Items
- Sales Report
- Logs
- Machines
- Users
- Customers
- Recyclable Items

This separation keeps product catalog management distinct from per-machine inventory management.

Input and Output

Main inputs:

- product selection
- cash/QR payment amount
- recycle point quantity
- RFID scan
- admin login credentials
- catalog item information
- inventory slot, stock, and capacity values
- machine name, address, and optional map coordinates

Main outputs:

- vending item display
- customer receipt
- stock updates
- admin dashboard metrics
- sales reports
- event logs
- RFID dashboard and eco-credit balance
- Arduino LCD and LED feedback

Error Messages and Feedback

The system provides feedback for:

- Supabase connectivity issues
- missing .env configuration
- invalid slot IDs
- duplicate machine slot assignment
- duplicate catalog item names
- insufficient money or eco-points
- unavailable items or empty stock
- RFID registration or lookup status
- QR payment status

6. Challenges and Solutions

Development Challenges

- Keeping the customer 12-slot UI aligned with normalized database slot IDs
- Separating global catalog data from machine-specific inventory data
- Preserving sales report history after a catalog item is deleted
- Designing the recycling reward flow so it encourages proper disposal without making the vending process complicated

- Keeping duplicate slots for the same item consistent within one machine
- Preventing slow Supabase/RFID operations from freezing the WPF UI
- Handling hardware responses quickly enough for Arduino feedback
- Supporting demo-friendly local images without depending on cloud storage

Problem-Solving

- items stores the shared product catalog, while machine_inventory stores per-machine slot state.
- Catalog deletion uses soft delete so active screens hide removed items while historical reports remain readable.
- Recyclable item definitions make eco-credit values configurable instead of hard-coded.
- Inventory validation enforces slots 1 through 12 and blocks duplicate slot assignment.
- Machine item price overrides are synchronized for the same item within the same machine.
- RFID lookup and registration operations run asynchronously before updating UI controls.
- ImageLoader provides safe fallback behavior for broken or missing image paths.
- SupabaseSessionCoordinator centralizes customer-mode online checks and persistence routing.

7. Testing

Test Cases

Manual test cases used for the current build include:

- launch app with valid .env
- launch app with missing or placeholder .env
- open customer mode with Supabase online
- select a vending machine and load its inventory
- buy an item with inserted cash
- buy an item with QR-paid balance
- buy an item with eco-points after RFID linking
- recycle items and save points to an RFID account
- scan a registered RFID card
- scan an unregistered RFID card and complete registration
- open admin mode and authenticate as an admin
- open inventory-manager mode and confirm assigned-machine restrictions
- add/edit/soft-delete a catalog item
- assign a catalog item to a machine slot
- restock an inventory item
- update machine item price override
- view sales reports and event logs
- display and print receipt information

Results

The tested flows confirm that the system supports its main demo requirements:

- customer vending works with stock updates
- sales and event logs are recorded
- receipts summarize item quantity, payment mode, cash/QR paid amount, eco-points used, and recycle points earned
- RFID cards can register customers and save eco-credits

- admin users can manage catalog, inventory, machines, users, customers, logs, and reports
- inventory managers are limited to assigned machine inventory
- deleted catalog items disappear from active use while old sales remain reportable

Limitations

- The project does not currently include a full automated test suite.
- Customer, admin, and RFID data features require internet access and live Supabase connectivity.
- Password fields are currently compared directly and should be hardened before production use.
- The customer UI is intentionally limited to 12 visible slots.
- Offline behavior is not fully implemented; the current build is Supabase-first.

8. Future Enhancements

Planned Features

- Stronger password hashing or Supabase Auth integration
- Stricter Supabase Row Level Security policies
- More complete offline mode for temporary disconnected operation
- Better telemetry support for ESP32 or vending machine sensors
- Clearer sustainability analytics, such as recycled item counts and estimated waste diverted
- Expanded reporting for sales forecasting and restock planning
- More automated test coverage for inventory, receipts, and RFID flows

Performance Improvements

- Move more long-running admin operations fully to async UI flows
- Add backend RPC helpers for multi-step writes that should behave transactionally
- Cache stable lookup data such as roles, recyclable items, and machine lists where appropriate
- Add more indexes based on actual Supabase query usage
- Improve image asset sizing for faster UI loading

9. Conclusion

Reflection

Eco-Matic demonstrates how a desktop application can connect user interface workflows, relational data design, cloud services, and physical hardware. The project moved beyond a simple CRUD application by adding machine-specific inventory, RFID customer accounts, QR payment simulation, receipts, sales reporting, and Arduino feedback. It also connects the system design to a community issue by encouraging recycling through credits that can be used for purchases.

Takeaways

The most important technical takeaways are:

- separating services from UI code makes the project easier to maintain
- normalized database design matters when the same product can exist in many machines
- hardware integration needs fast and clear response handling
- good documentation helps connect the running app, code structure, ERD, and class diagrams
- SDG-based design can make a software project more meaningful by connecting technical features to real environmental behavior
- known limitations should be documented clearly so future improvements have a realistic direction

Appendix

Source Code

GitHub repository:

```
https://github.com/SeanixReal/Eco-Matic-WPF-Supabase
```

Related Project Documents

- README.md
- docs/README.md
- docs/CODEBASE_ARCHITECTURE.md
- docs/DIAGRAMS.md
- docs/diagrams/ERD.md
- docs/diagrams/SIMPLIFIED_CLASS_DIAGRAM.md
- docs/diagrams/FULL_CLASS_DIAGRAM.md
- docs/USER_MANUAL.md
- docs/SUPABASE_AUDIT.md
- docs/CODE_REVIEW.md

References

- Microsoft .NET and WPF documentation
- United Nations Sustainable Development Goal 12: Responsible Consumption and Production
- Supabase PostgreSQL, PostgREST, and Edge Function documentation
- Arduino serial communication documentation
- RC522 RFID reader references
- QRCoder package documentation
- WebView2 documentation